

JEPA-Inspired Latent Representation Routing for Domain-Specialized LoRA Experts in Large Language Models

A Proof-of-Concept Experimental Research Study

Author: Elias Tadele

Date: 21/08/2026

Terminology note. The implemented system is an adapter-level mixture-of-LoRA system. It is not a conventional token-level sparse Transformer Mixture-of-Experts model such as Switch Transformer or Mixtral. The paper therefore uses “JEPA-inspired routing,” “domain-specialized LoRA experts,” and “adapter-level expert routing” for the present implementation. A true sparse Transformer MoE is treated as future work.

Abstract

Automated LLM judging via Gemini was planned but the external API quota was exhausted before it could run. To avoid leaving generation quality unevaluated, all 20 examples were subsequently scored by Claude (Anthropic) on the same three-axis rubric (correctness, relevance, completeness, 1-10), a substitution disclosed explicitly in Section 6.4. Under this scoring, mean quality was 8.60/10 for Gemini, 5.33/10 for the raw base model, 4.93/10 for the combined fine-tune, and 2.18/10 for the JEPA-routed MoE-LoRA system, the lowest of the four. Qualitative inspection indicates the low MoE-LoRA and combined-fine-tune scores are driven substantially by topic drift and memorized-content regurgitation rather than by routing error alone.

Keywords: large language models; JEPA; LoRA; parameter-efficient fine-tuning; expert routing; mixture of experts; adapter composition; latent representation learning.

1. Introduction

1.1 Background

The Transformer architecture established a highly parallelizable attention-based foundation for modern language modeling [1]. Subsequent scaling of Transformer language models has produced systems with broad knowledge and strong coding, reasoning, and instruction-following capabilities. The Qwen2.5 family, for example, reports substantial improvements from large-scale pretraining and post-training and includes specialized coding variants relevant to the present prototype [2]. Yet a single general-purpose model remains a difficult fit for applications that contain sharply different technical subdomains. A question about retrieval indexing, for example, may require different conventions and explanatory priorities from a question about multi-agent orchestration or statistical analysis.

One response is to maintain a separate fully fine-tuned model for each domain. This is often impractical because each model duplicates the base parameters, increases storage requirements, and complicates deployment. Parameter-efficient fine-tuning addresses this problem by freezing the shared model and learning a small adaptation module. LoRA represents an update to a model weight matrix through a low-rank factorization, greatly reducing trainable parameters while preserving a common pretrained backbone [3]. Quantized variants such as QLoRA further reduce memory requirements by backpropagating through a frozen quantized model into LoRA parameters [4].

A modular adapter collection creates a second problem: **expert selection**. Given a query and several domain-specific adapters, the system must decide which adapter—or which combination of adapters—should be applied. Conventional Mixture-of-Experts systems address an analogous problem with a learned router that conditionally activates expert submodules. Switch Transformer uses simplified top-1 routing, while Mixtral routes each token to two feed-forward experts at every layer [5] [6]. These architectures are not identical to the present system: the prototype routes among complete LoRA adapters outside the token-level Transformer computation rather than replacing internal feed-forward blocks with sparse experts.

The research direction explored here combines this adapter-routing problem with the central idea of Joint-Embedding Predictive Architectures (JEPAs): predicting a target representation from a contextual representation in latent space rather than reconstructing the original input [7]. I-JEPA was developed for image representation learning, where a context block predicts representations of target blocks. The present work adapts this broad principle to language-derived embeddings and therefore calls the method **JEPA-inspired language representation learning**, not an exact reproduction of I-JEPA.

1.2 Problem statement

A shared LLM may contain useful knowledge across several technical domains, but its responses can vary in relevance, terminology, and structure depending on the domain of the query. Domain-specialized LoRA adapters offer a resource-efficient way to encode different response tendencies. The central problem is then to map a query to the adapter that is most likely to produce a domain-aligned answer.

The prototype compares two routing representations. The first is a conventional Sentence Transformer semantic representation used as a baseline. The second is a learned latent representation produced by a JEPA-inspired context encoder trained to predict a target sentence representation. The study asks whether the latter representation yields a better geometry for domain-level expert selection.

1.3 Motivation and research gap

The relevant literature establishes the value of Transformers, sparse expert routing, LoRA, quantized adaptation, and predictive latent representation learning. More recent work has also examined routed parameter-efficient fine-tuning and argues that adaptation modules should be aligned with multi-expert architectures [8]. However, the supplied prototype addresses a distinct and narrower configuration: a single small shared LLM, several separately trained LoRA adapters, a representation-level router, and a JEPA-inspired predictor trained from sentence-embedding context-target pairs.

The contribution is therefore not a claim of state-of-the-art performance and not a claim that JEPA improves LLM quality. Instead, the study provides a controlled proof-of-concept investigation of whether predictive latent learning can be used as a routing representation. The negative result is central: under the current configuration and small evaluation subset, the proposed router performs below the Sentence Transformer baseline.

1.4 Objectives

The general objective is to investigate whether JEPA-inspired latent representation learning can be used as an effective routing mechanism for domain-specialized LoRA experts in a large language model. Specific objectives are to construct five domain-specialized LoRA experts around a shared LLM; train a JEPA-inspired latent prediction module; integrate that representation into expert routing; compare it with a semantic embedding baseline; measure routing accuracy; inspect routed generations qualitatively; identify architectural limitations; and propose a stronger future JEPA-MoE design.

1.5 Research questions and hypothesis

The study is organized around five research questions:

1. Can the implemented JEPA-inspired objective be optimized on language-derived context-target representations? 2. Can the resulting latent representation support domain-level expert routing? 3. Does JEPA-inspired routing outperform a conventional Sentence Transformer router on the available evaluation subset? 4. What observable routing and generation failure modes appear in the raw outputs? 5. What architectural changes are required to create a stronger integrated JEPA-MoE system?

The original hypothesis was:

H1: A JEPA-inspired latent representation can provide a more effective domain-routing signal than a conventional sentence-embedding representation.

The current experiment does **not** support H1 under the evaluated configuration. This conclusion is limited to the implementation, data, evaluation subset, and training procedure reported here.

1.6 Scope and significance

This is an exploratory proof-of-concept study conducted under constrained computational resources. It uses Qwen2.5-Coder-1.5B-Instruct, five LoRA experts, 1,458 domain examples, 1,264 JEPA context-target pairs, and a 20-example routing evaluation. It does not provide a statistically powered benchmark, multiple-seed analysis, automated generation scoring, or a conventional sparse Transformer MoE. Its significance lies in making the methodological combination explicit, preserving the negative result, and identifying where the current architecture does and does not provide evidence.

1.7 Paper organization

Section 2 reviews the relevant literature. Section 3 defines the technical foundations and distinguishes conventional MoE from adapter-level routing. Section 4 describes the prototype methodology and implementation. Section 5 presents the experimental design. Section 6 reports quantitative and qualitative results. Section 7 discusses interpretation and failure modes. Section 8 states limitations and reproducibility constraints. Section 9 proposes future work, and Section 10 concludes.

2. Literature review

2.1 From Transformers to conditional computation

The Transformer introduced self-attention as the principal sequence-processing mechanism and demonstrated improved translation quality with greater parallelism than recurrent alternatives [1]. Modern LLMs inherit this architecture but add large-scale pretraining, instruction tuning, and domain-specific post-training. The Qwen2.5 technical report describes a family trained with substantially expanded pretraining data and extensive supervised and reinforcement-learning post-training, including specialized coding models [2]. The present project uses a small coding-oriented Qwen2.5 model as a shared foundation rather than attempting to train a new language model.

Conditional computation is a complementary response to model scaling. Dense models apply the same parameters to all inputs, whereas MoE models activate only a subset of experts. Switch Transformer simplifies the router and uses top-1 expert selection, reporting improved scaling efficiency under matched computational resources [5]. Mixtral uses eight feed-forward experts and selects two experts per token, demonstrating that a model can expose a much larger total parameter count while using fewer active parameters per token [6]. These results establish the relevance of routing, but also show that routing is coupled to expert architecture, load balancing, communication, and training stability.

2.2 Parameter-efficient adaptation and modular experts

Adapters insert small trainable modules into a frozen pretrained model, enabling task-specific adaptation without duplicating all model parameters [9]. LoRA takes a particularly simple approach: it freezes the original weights and learns a low-rank update for selected matrices [3]. If the original matrix is W , the adapted matrix is

$$W' = W + \Delta W, \quad \Delta W = BA,$$

where A and B are trainable low-rank matrices. QLoRA combines this strategy with 4-bit quantization and memory-saving techniques, showing that large models can be adapted under substantial hardware constraints [4]. In the present work, LoRA is used to create five separate domain experts around one frozen base model.

This modularity creates opportunities for adapter composition and routing, but it also creates a mismatch if the routing signal is learned independently of the downstream objective. A semantic embedding can place queries near domain centroids without changing the LLM. A learned latent space may be more task-specific, but only if its training objective preserves the information needed for domain discrimination. The current study tests this tension directly.

2.3 JEPA and predictive latent representation learning

I-JEPA learns image representations by predicting target-block representations from a context block. Its central design is non-generative: the model predicts in representation space rather than reconstructing pixels. The authors emphasize that target scale and context structure are important for semantic representation learning [7]. The present implementation borrows the latent prediction principle but changes the modality, data construction, architecture, and downstream purpose. It begins with normalized Sentence Transformer embeddings, constructs a mean context embedding by removing one randomly selected sentence, and trains an encoder-predictor-target-encoder system.

This distinction matters methodologically. The prototype is **JEPA-inspired** because it contains context and target representations, a predictor, a momentum-updated target encoder, a cosine-alignment term, and variance regularization. It is not an exact I-JEPA reproduction: it does not operate directly on image patches, does not use the I-JEPA masking strategy, and does not train directly on raw LLM hidden states.

A predictive latent-representation objective of this kind is vulnerable to representational collapse: without a countermeasure, a context encoder can trivially minimize a prediction loss by mapping every input to a constant or near-constant vector, since a constant is easy for the target encoder to match when the target encoder is initialized identically and updated only by exponential moving average. The implementation's variance penalty (Section 3.4) is a direct, if simplified, application of the variance-preservation principle used in variance-invariance-covariance regularization for self-supervised representation learning, which penalizes the encoder when the per-dimension standard deviation of its output across a batch falls below a fixed threshold [10]. This is why the JEPA-inspired objective's convergence to a stable negative value (Table 5, Figure 3) is evidence that the encoder optimized its stated objective without collapsing to a degenerate constant output, but it remains, as Section 3.4 notes, evidence of successful optimization rather than evidence of routing usefulness; the two are empirically distinct, and Section 6.2 measures the latter directly.

2.4 Routed adaptation and the position of this study

Recent routed fine-tuning work studies how parameter-efficient adaptation interacts with models that already contain multiple experts. Liu et al. report experiments on OLMoE and Mixtral and argue that PEFT should account for the underlying expert routing dynamics [8]. Their setting is closer to conventional MoE adaptation than the present prototype, because the base models themselves contain internal MoE structure. The result nevertheless reinforces the broader methodological point that adaptation and routing cannot be treated as entirely independent components.

The present study occupies a smaller but useful design point. It keeps the base LLM dense and shared, trains five modular LoRA adapters, and applies routing at the adapter level. The JEPA-inspired representation is not trained with

an explicit domain-classification loss, so the experiment tests whether predictive latent alignment transfers to routing—not whether a supervised router can classify domains.

Study	Year	Method	Architecture	Dataset / evaluation	Main result	Limitation relative to this study
Vaswani et al. [1]	2017	Self-attention Transformer	Dense encoder-decoder	WMT translation and parsing	Strong quality with parallel training	No modular expert routing
Fedus et al. [5]	2022	Switch Transformer	Token-level sparse top-1 MoE	C4 and multilingual settings	Improved scaling efficiency	Requires internal Transformer experts and balancing
Hu et al. [3]	2021	LoRA	Frozen model plus low-rank updates	GLUE and language-model tasks	Competitive adaptation with fewer trainable parameters	No representation-based multi-adapter router
Dettmers et al. [4]	2023	QLoRA	Quantized frozen model plus LoRA	Instruction tuning and chatbot evaluation	Large memory savings with strong quality	Focuses on adaptation efficiency, not routing
Assran et al. [7]	2023	I-JEPA	Context encoder, predictor, target encoder	ImageNet and downstream vision tasks	Semantic latent representation learning	Visual architecture; not language adapter routing
Jiang et al. [6]	2024	Mixtral	Token-level top-2 sparse MoE	Language, mathematics, coding, multilingual tasks	Strong quality with sparse activation	Not adapter-level routing
Qwen et al. [2]	2025	Qwen2.5 family	Dense and specialized LLM variants	Broad language, reasoning, and coding benchmarks	Strong open-weight model family	Base-model report, not a routing method
Liu et al. [8]	2026	Parameter-efficient routed fine-tuning	PEFT applied to MoE LLMs	OLMoE and Mixtral tasks	Studies alignment of adaptation with MoE routing	Different base architecture and evaluation setting
This study	2026	JEPA-inspired adapter routing	Shared dense LLM plus five LoRA experts	20-example domain-routing subset	80% vs. 90% baseline accuracy	Small exploratory evaluation and no joint routing loss

Based on this literature, the unresolved question is not whether Transformers, LoRA, MoE, or JEPA are individually useful. It is whether a predictive latent space trained from contextual language representations can serve as a reliable boundary between domain-specialized adapter experts. The present work treats that question as empirical and reports the answer conservatively.

3. Theoretical and technical background

3.1 LoRA experts

Let $W \in \mathbb{R}^{d_o \times d_i}$ be a frozen model weight matrix. LoRA learns $A \in \mathbb{R}^{r \times d_i}$ and $B \in \mathbb{R}^{d_o \times r}$, where $r \leq \min(d_i, d_o)$. The resulting update is $\Delta W = BA$, often scaled by α/r . In the prototype, each domain has its own collection $\{A_k, B_k\}$, while the Qwen base weights are shared. The five adapters therefore represent modular changes in behavior rather than five independent full LLMs.

3.2 Conventional Mixture-of-Experts

A conventional MoE layer contains experts $E_1, \dots, E_K$ and a gating function $g(x)$. A dense formulation can be written as

MoE output: $y = \sum_i g_i(x) \cdot E_i(x)$, with $\sum_i g_i(x) = 1$.

Sparse MoE uses top-k routing, so only a subset of experts processes each token or example. The router is commonly trained together with the experts, and auxiliary objectives may encourage balanced expert utilization. In Switch Transformer and Mixtral, the experts are internal Transformer submodules and the routing decision can change across tokens and layers [5] [6].

A further practical concern in conventional sparse MoE training is load balancing: without an explicit auxiliary loss encouraging even utilization, a router can collapse onto a small subset of experts, under-training the rest and wasting the additional capacity the architecture was meant to provide. Switch Transformer and related work address this with an auxiliary load-balancing loss added to the primary training objective [5]. The present system does not face an identical risk in the same form, because its five LoRA experts are trained independently and completely before any routing decision is made, so there is no expert that can be starved of gradient updates during training. The analogous risk here is a different one: a routing representation that fails to separate domains can cause the system to behave as if it had fewer effective experts than the five that were actually trained, since a large share of queries collapse onto the same one or two adapters regardless of their true domain. The near-uniform top-2 route weights recorded in the raw outputs (Section 6.2.1, Figure 6) are consistent with a mild version of this at inference time, though the present data cannot distinguish it from genuine cross-domain ambiguity in the queries themselves.

3.3 Difference between conventional MoE and the present system

The distinction is essential:

Conventional sparse Transformer MoE	Present adapter-level system
Router usually operates on hidden states inside Transformer layers	Router operates on an external query representation
Experts are internal feed-forward or Transformer submodules	Experts are separately trained LoRA adapters
Routing can occur per token and per layer	Routing is performed at query level
Expert outputs are combined within the layer	Selected adapter weights are combined, then used by the shared LLM
Router and experts are commonly trained jointly	JEPA representation and LoRA experts are trained in separate stages

The prototype’s inference pathway is therefore

Inference path: query $\rightarrow$ representation $\rightarrow$ router $\rightarrow$ LoRA composition $\rightarrow$ shared LLM $\rightarrow$ response.

Calling this a “mixture-of-LoRA experts” system is technically appropriate. Calling it a full sparse Transformer MoE would be inaccurate.

3.4 JEPA-inspired language formulation

For a context c and target t , the prototype begins with normalized Sentence Transformer embeddings $e(c)$ and $e(t)$. The context is formed as the mean of all sentence embeddings except one randomly selected target sentence. The context encoder f_θ and target encoder $f_{\tilde{\theta}}$ map these vectors to latent representations:

Context and target latents: $z_c = f_\theta(e(c))$ and $z_t = f_{\tilde{\theta}}(e(t))$.

The predictor g_ϕ estimates the target latent representation:

Predicted target latent: $\hat{z}_t = g\phi(z_c)$.

The implemented prediction term is negative cosine similarity:

Prediction loss: $L_{\text{pred}} = -(1/n) \sum_j \text{cosine}(\hat{z}_{t,j}, z_{t,j})$.

The implementation adds a variance penalty on the context latent dimensions:

Variance penalty: $L_{\text{var}} = (1/d) \sum_m \max(0, 1 - \sqrt{\text{Var}(z_{c,m}) + 10^{-4}})$.

The total objective is

Prediction loss: $L_{\text{pred}} = -(1/n) \sum_j \text{cosine}(\hat{z}_{t,j}, z_{t,j})$.

The target encoder is initialized from the context encoder and updated using exponential moving average with decay 0.996. Only the context encoder and predictor receive gradient updates. Because the prediction term is negative cosine similarity, a negative total objective is not inherently problematic and cannot be interpreted as a universal representation-quality score.

3.5 Routing function

The baseline router embeds a query and compares it with domain representations. The JEPA router embeds the query using the same Sentence Transformer, applies the trained context encoder, and compares the resulting vector with the mean encoded representation of each domain. For domain k , the score is cosine similarity:

Routing score for domain k : $s_k(q) = \text{cosine}(f\theta(e(q)), \mu_k)$.

where μ_k is the mean latent representation of domain k . For top- K composition, the notebook converts the selected scores to weights with a softmax:

Top- k adapter weight: $w_k = \exp(s_k) / \sum_{j \in \text{top-}k} \exp(s_j)$.

The prototype uses top-2 weighted adapter composition for generation, while the reported routing accuracy uses top-1 selection.

Routing confidence can be quantified directly from the top- k weights the router already produces. For a query routed to weights $\{w_1, \dots, w_k\}$, define routing entropy as $H(w) = -\sum_i w_i \log_2(w_i)$, measured in bits. For top-2 composition, $H(w)$ reaches its maximum of 1.0 bit exactly when $w_1 = w_2 = 0.5$, meaning the router is maximally undecided between its two top choices, and approaches 0 bits as one weight dominates, meaning the router is effectively making a confident top-1 decision even though it is nominally blending two experts. This quantity is computed directly from the recorded route weights in Section 6.2.2 and used to test whether the router's blending behavior is confidence-weighted in practice or close to a fixed, uninformative split.

A second mathematical consideration concerns what the weighted composition in Section 3.5 actually computes. Let $f(x; W)$ denote the base model's output for input x under parameters W , and let $\delta W_1, \delta W_2$ denote two experts' LoRA updates. The system computes $f(x; W_0 + w_1 \delta W_1 + w_2 \delta W_2)$, a single forward pass through parameters that are a linear combination of two adapters' weights. This is different in kind from computing $w_1 f(x; W_0 + \delta W_1) + w_2 f(x; W_0 + \delta W_2)$, a linear combination of two experts' outputs. The two are equivalent only if f is linear in W , which it is not: f is a deep composition of attention and feed-forward nonlinearities. Weight-space averaging of independently fine-tuned models has been shown to preserve or improve behavior in some settings, most notably when the models share an initialization and are fine-tuned toward compatible objectives [11], but that literature typically reports results for carefully selected checkpoint combinations rather than arbitrary similarity-weighted pairs, and does not establish that the merged parameters behave as an interpolation of the two experts' individual behaviors in the general case. Whether the present system's weight-space merge preserves, degrades, or

unpredictably alters each expert's individual behavior is not directly measured in this study and is treated in Section 7.2c as one of three candidate mechanisms behind the generation-quality results in Section 6.4.

4. Proposed methodology

4.1 Research design

The study is an exploratory proof-of-concept experimental comparison. The independent variable is the routing representation: Sentence Transformer embedding versus JEPA-inspired latent representation. The dependent variable is top-1 domain-routing accuracy. The selected LoRA experts and shared base model are held conceptually constant across the routing comparison.

4.2 System architecture

The complete conceptual pipeline is shown in Figure 2. The figure makes explicit that the shared base LLM is combined with five external LoRA adapters rather than replaced by token-level Transformer experts.

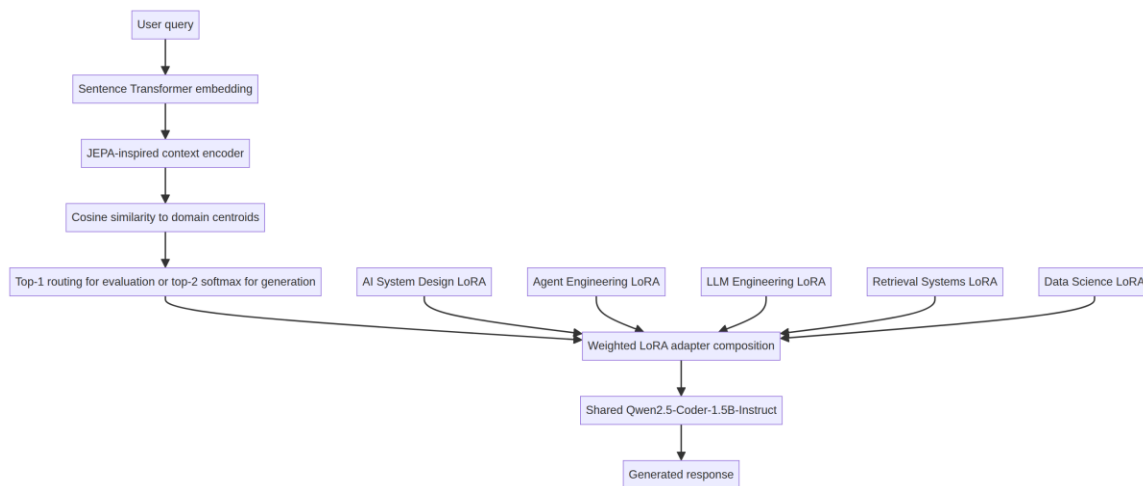


Figure 2. Overall architecture of the JEPA-inspired adapter-routing system.

4.2.1 Model operation in plain language

The model can be understood as a shared specialist system. The Qwen base model is the common language engine, while each LoRA adapter supplies a small domain-specific adjustment. The router does not rewrite the query and does not choose a new full model. Instead, it estimates which adapter is most relevant.

The proposed route has five practical steps. First, the query is converted into a normalized sentence embedding. Second, the JEPA-inspired encoder maps that embedding into a smaller 64-dimensional latent space. Third, the encoded query is compared with one centroid for each domain. Fourth, the system selects the highest-scoring domain for routing evaluation or the two highest-scoring domains for generation. Finally, the selected LoRA updates are applied to the shared Qwen model, which generates the answer. This separation is useful because it distinguishes the decision problem—selecting an expert—from the generation problem—producing a technically correct response.

The complete conceptual pipeline is:

The shared base model contains approximately 1.562 billion parameters. The five adapters are independent domain-specific parameter updates. The router does not replace the LLM’s internal computation with five Transformer experts; it selects or composes the external adaptation modules.

4.3 Dataset construction

The domain corpus contains 1,458 instruction-style examples distributed as follows.

Domain	Examples	Share of corpus
AI System Design	400	27.4%
Agent Engineering	260	17.8%
LLM Engineering	289	19.8%
Retrieval Systems	300	20.6%
Data Science	209	14.3%
Total	1,458	100.0%

The examples were formatted into instruction, optional input, and response fields. The available materials do not provide a complete independent train/validation/test protocol for each expert, so the study reports the corpus counts but does not claim a fully controlled generalization benchmark.

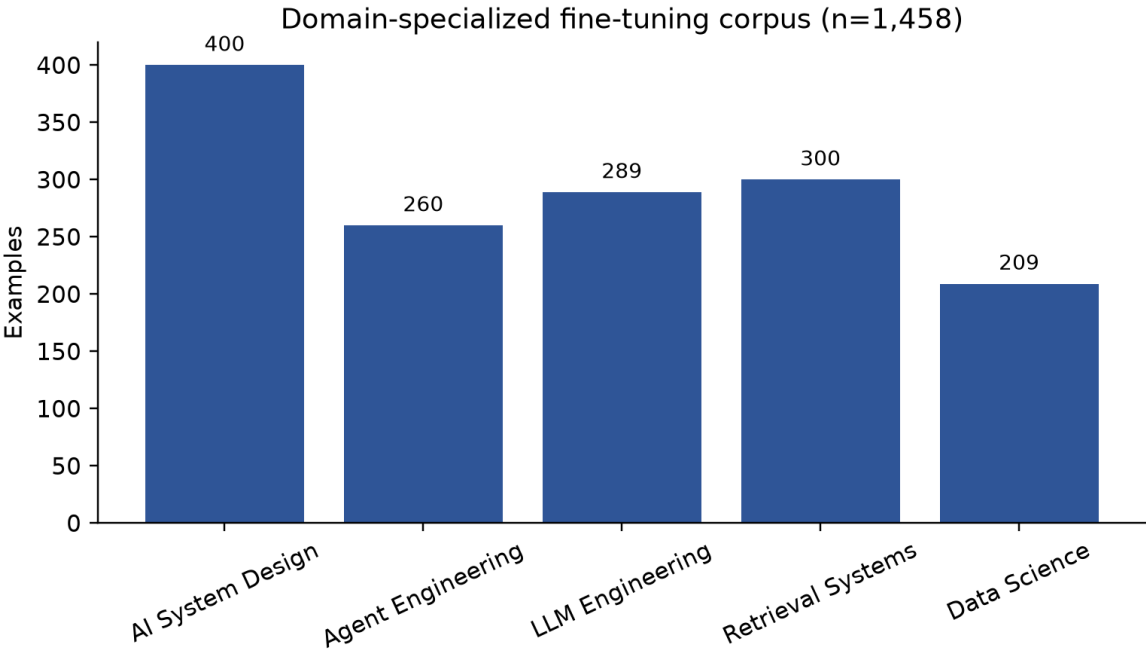


Figure 1. Distribution of the domain-specialized fine-tuning corpus.

4.4 Expert construction and LoRA configuration

A fresh Qwen2.5-Coder-1.5B-Instruct model was loaded for each domain and adapted with LoRA using Unsloth and PEFT-compatible components. The notebook reports three training epochs, a per-device batch size of 2, gradient accumulation of 4, effective batch size 8, learning rate 2×10^{-4} , LoRA rank 16, alpha 32, dropout 0.0, sequence length 2048, 4-bit loading, and top-2 adapter composition at inference. The logged trainable parameter count was 18,464,768 out of 1,562,179,072, or 1.18%.

The notebook’s visible configuration specifies the LoRA hyperparameters and quantized loading. It does not provide sufficient evidence for a reliable statement about the optimizer, scheduler, random seed for the LoRA stage, or complete target-module list in the final report. Those items are therefore marked **not fully documented** rather than inferred.

Component	Verified configuration
Base model	unsloth/Qwen2.5-Coder-1.5B-Instruct
Quantization	4-bit loading
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.0
Learning rate	2×10^{-4}
Epochs	3
Device batch size	2
Gradient accumulation	4
Effective batch size	8
Maximum sequence length	2048
Trainable parameters	18,464,768
Total parameters	1,562,179,072
Trainable fraction	1.18%
GPU	Tesla T4
LoRA optimizer / scheduler	Not fully documented in supplied materials

4.5 JEPA-inspired training

The JEPA dataset contains 1,264 context-target pairs. For every eligible instruction-response example, the implementation splits the concatenated instruction and output into sentences, requires at least three sufficiently long sentences, randomly selects one target sentence using seed 42, and averages the remaining sentence embeddings into a context vector. Sentence Transformer embeddings are normalized before entering the JEPA module.

The context and target encoders are two-layer multilayer perceptrons with a 256-unit hidden layer, GELU activation, and 64-dimensional output. The predictor maps 64 dimensions through a 128-unit hidden layer back to 64 dimensions. The context encoder and predictor are optimized using Adam with learning rate 10^{-3} , batch size 64, and 40 epochs. The target encoder is updated by exponential moving average rather than gradient descent.

4.6 Baseline and proposed routers

The baseline router uses the original Sentence Transformer representation and domain similarity. The proposed router applies the learned JEPA context encoder to both query embeddings and domain-corpus embeddings. It computes cosine similarity to the five domain centroids, ranks the domains, and either selects the top domain for accuracy evaluation or forms a weighted top-2 adapter combination for generation.

4.7 Inference pipeline

At inference, a query is converted to a Sentence Transformer vector. In the proposed route, the vector is passed through the JEPA context encoder. The router computes similarity against the five domain centroids. For routing evaluation, the highest-scoring domain is compared with the annotated domain. For generation, the top two domains are converted into normalized weights, the corresponding LoRA adapters are combined linearly, and the shared Qwen model generates deterministically with a maximum of 256 new tokens in the notebook's generation function.

4.8 Reproducibility

The JEPA stage explicitly sets Python, NumPy, and PyTorch seeds to 42. The LoRA training stage does not have equally complete seed documentation in the supplied materials. The data files themselves are not included as separate attachments, although the notebook reports their counts. Checkpoint paths are recorded in the notebook as

/content/experts/{domain}. The software environment includes Unsloth, Sentence Transformers, scikit-learn, pandas, Matplotlib, PEFT, Transformers, and PyTorch, but exact package lock files are unavailable. These omissions matter because routing accuracy on only 20 examples can vary with data ordering, checkpoint versions, and random initialization.

4.9 Capacity relative to per-domain data volume

Each expert has 18,464,768 trainable LoRA parameters (Table 3) but is trained on a domain corpus ranging from 209 to 400 examples (Table 2). With effective batch size 8 (per-device batch 2, gradient accumulation 4) and 3 epochs, the total number of optimizer steps per expert ranges from 81 (data_science, 209 examples) to 150 (ai_system_design, 400 examples), computed as $\text{ceil}(n_{\text{examples}} / 8) * 3$. Table 11 reports both quantities per domain alongside the corresponding trainable-parameters-per-example ratio.

Domain	Training examples	Optimizer steps (3 epochs)	Trainable params per example
data_science	209	81	88,348
agent_engineering	260	99	71,018
llm_engineering	289	111	63,892
retrieval_systems	300	114	61,549
ai_system_design	400	150	46,162

Table 11. Per-domain training volume, optimizer steps, and trainable-parameter-to-example ratio.

These step counts are low relative to typical LoRA fine-tuning practice, which raises two competing candidate explanations for the generation-quality results rather than one. A high parameter-to-example ratio is classically associated with memorization risk; a low step count over few, stylistically homogeneous documentation-style examples is more consistent with shortcut learning, in which a model latches onto superficial, easily-fit surface patterns, such as document heading structure, rather than learning the intended task [12]. Both mechanisms predict the same observable symptom, fluent output that does not address the prompt, and the present data cannot fully separate them; Section 7.2c treats this explicitly as an open question rather than resolving it.

5. Experimental design

The primary comparison uses the first 20 rows of the evaluation file, as specified by `eval_rows = load_jsonl(CONFIG["eval_path"])[ :20]`. Each row contains a prompt and domain label. The baseline and JEPA routers are evaluated with top-1 selection against that domain label.

Experimental element	Definition
Independent variable	Routing representation
Baseline	Sentence Transformer representation
Proposed method	JEPA-inspired latent representation
Shared model	Qwen2.5-Coder-1.5B-Instruct
Experts	Five domain-specialized LoRA adapters
Primary metric	Top-1 domain-routing accuracy
Evaluation subset	20 examples
Generation evaluation	Qualitative inspection of raw outputs

Experimental element	Definition
Automated scoring	Completed using Claude as a disclosed substitute judge (n=20); Gemini quota exhausted

The sample is too small for strong statistical generalization. With 20 examples, each example changes accuracy by five percentage points. The reported 90% versus 80% result should therefore be treated as a preliminary measurement rather than a definitive estimate of population performance.

6. Results

6.1 JEPA-inspired objective

The reported objective changed sharply during the first training interval and then stabilized within a narrower range.

Epoch	Objective
0	0.2410
5	−0.9711
10	−0.9295
15	−0.9256
20	−0.8988
25	−0.8912
30	−0.9021
35	−0.8969
39	−0.9031

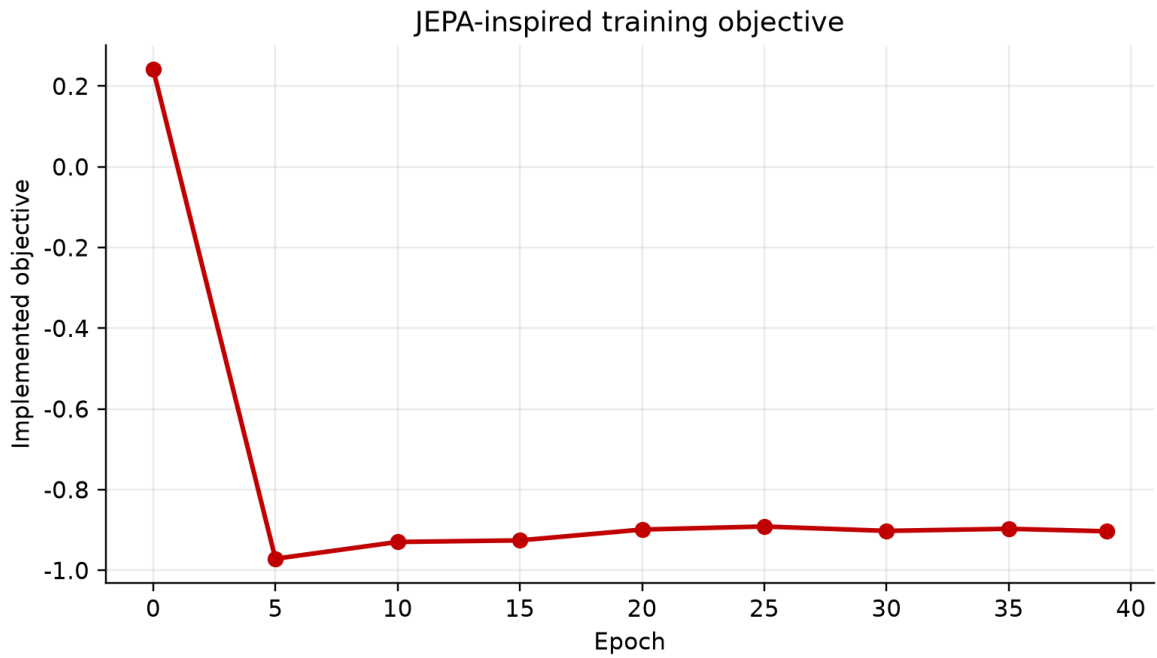


Figure 3. Evolution of the implemented JEPA-inspired objective.

The curve is consistent with optimization of the implemented objective: a rapid early change followed by stabilization. It does not establish that the learned representation is useful for routing. The objective combines negative cosine alignment with a variance term, so its negative scale is specific to this loss definition and should not be compared directly with unrelated representation-learning losses.

6.2 Routing accuracy

Router	Evaluation examples	Top-1 accuracy
Sentence Transformer baseline	20	90%
JEPA-inspired router	20	80%

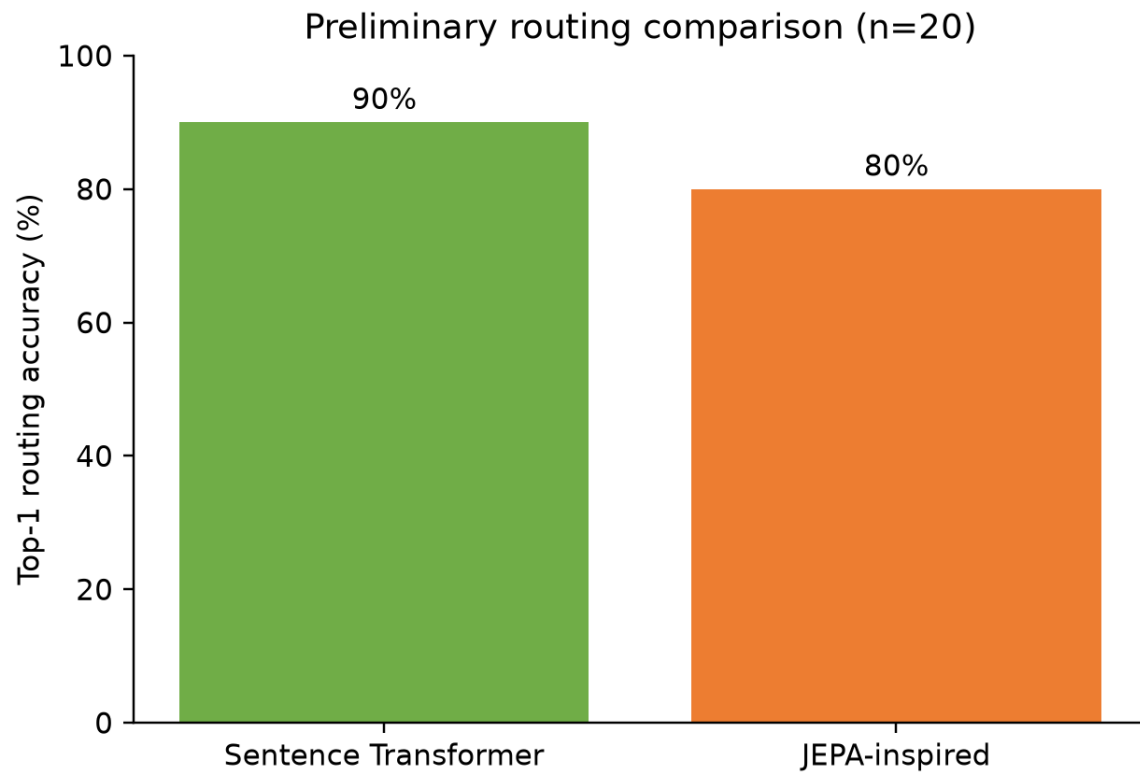


Figure 4. Preliminary top-1 domain-routing comparison.

6.2.1 Confusion structure and route confidence

The raw output file records the JEPA-inspired top-2 route for each of the 20 evaluated examples. Those recorded routes allow a JEPA-specific confusion matrix to be reconstructed. The matrix should not be interpreted as a baseline confusion matrix: the per-example baseline predictions were not included in the CSV. The row and column totals nevertheless make the main error pattern visible. The largest confusions are between AI System Design and Agent Engineering, while LLM Engineering and Data Science are separated more cleanly in this small subset.

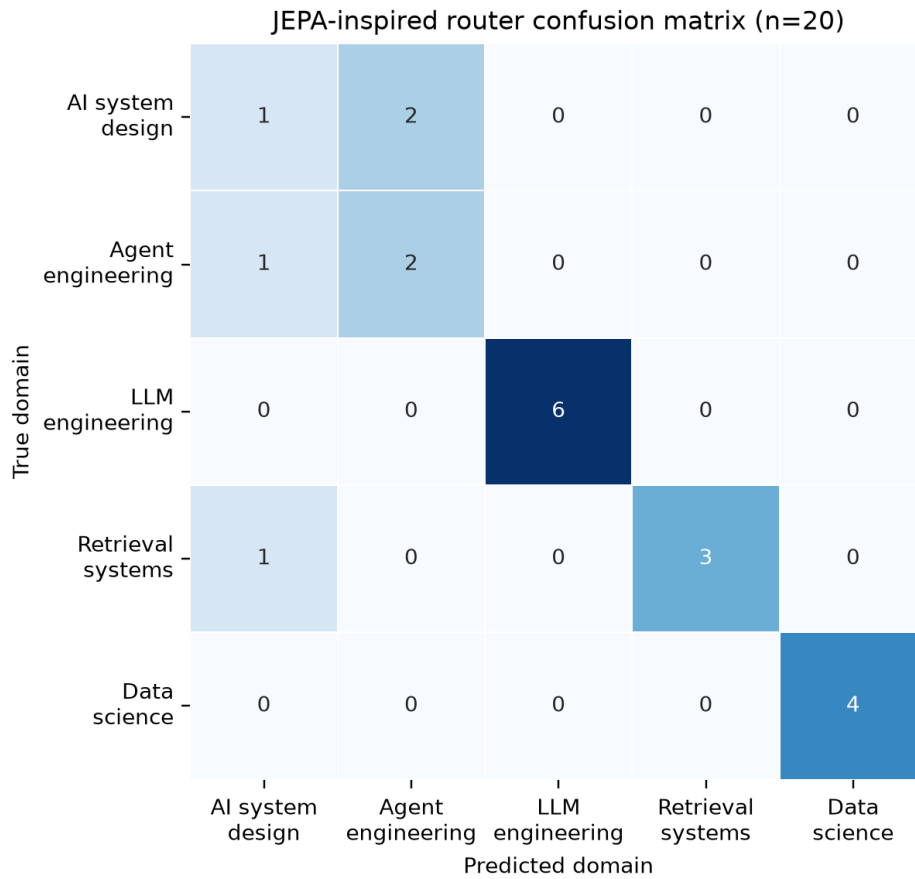


Figure 5. Confusion matrix for the recorded JEPA-inspired top-1 routes.

Figure 6 shows the top-1 route weight for each example. Several values are close to 0.50, which means that the top two adapters receive almost equal weight after the softmax transformation. These near-ties are consistent with uncertain domain boundaries and help explain why top-2 adapter composition may blend related but non-identical behaviors. Red bars indicate examples where the recorded top-1 route does not match the annotated domain; blue bars indicate matches.

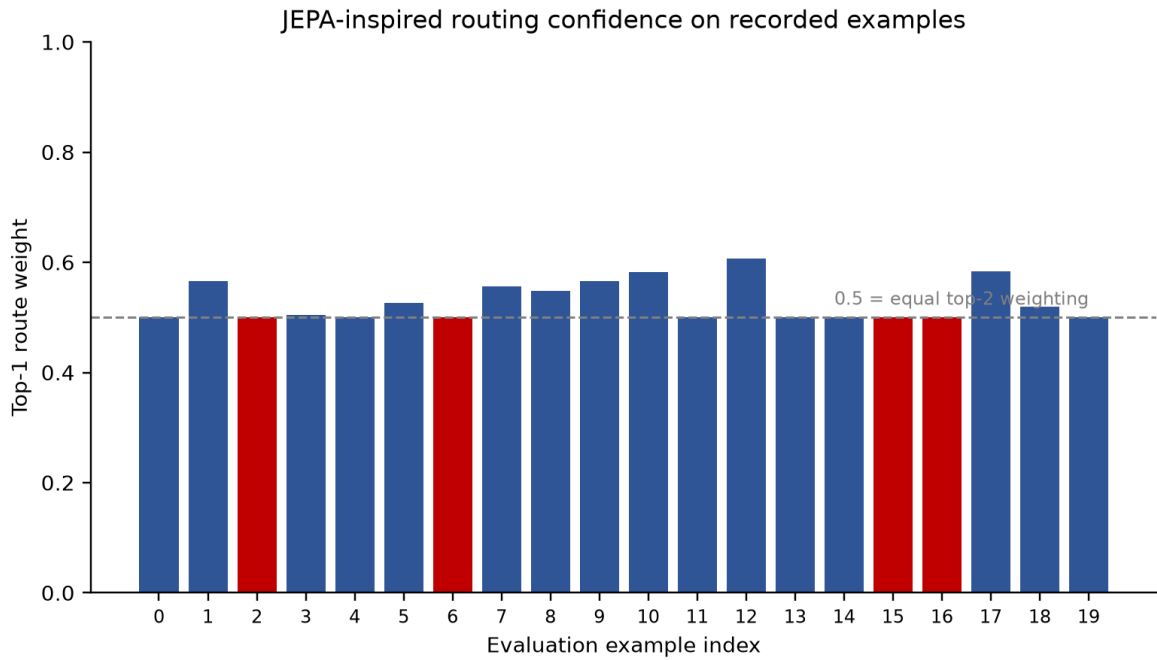


Figure 6. JEPA-inspired top-1 route weight for each recorded evaluation example.

The absolute difference is

80%-90%=-10 percentage points.

The relative difference with respect to the baseline is

$(80-90/90)*100 = -11.11\%$.

Under the current configuration and evaluation subset, the JEPA-inspired router did not outperform the baseline. The current result does not support H1. It does not demonstrate that JEPA is unsuitable for routing in general, because the experiment did not compare alternative JEPA objectives, larger datasets, multiple seeds, supervised routing losses, or direct hidden-state representations.

6.2.2b Is the router's blending confidence-weighted in practice?

Applying the entropy measure defined in Section 4.8 to the 20 recorded top-2 route-weight pairs shows that the router's blending sits close to a fixed 50/50 split for most examples rather than varying with how confidently the router separated the top two domains. Mean route entropy is 0.994 bits out of a maximum possible 1.0 bit (Figure 12), 10 of 20 examples have their top weight within 0.001 of an exact 0.5/0.5 tie, and the median gap between the top two weights across all 20 examples is 0.005. This means the intended design property of Section 3.5, in which the router's relative confidence between its top two choices determines how much each expert contributes, holds only weakly in practice at this scale: the second-ranked domain contributes nearly as much as the first-ranked one in most cases, regardless of how much more similar the top domain's centroid was to the query.

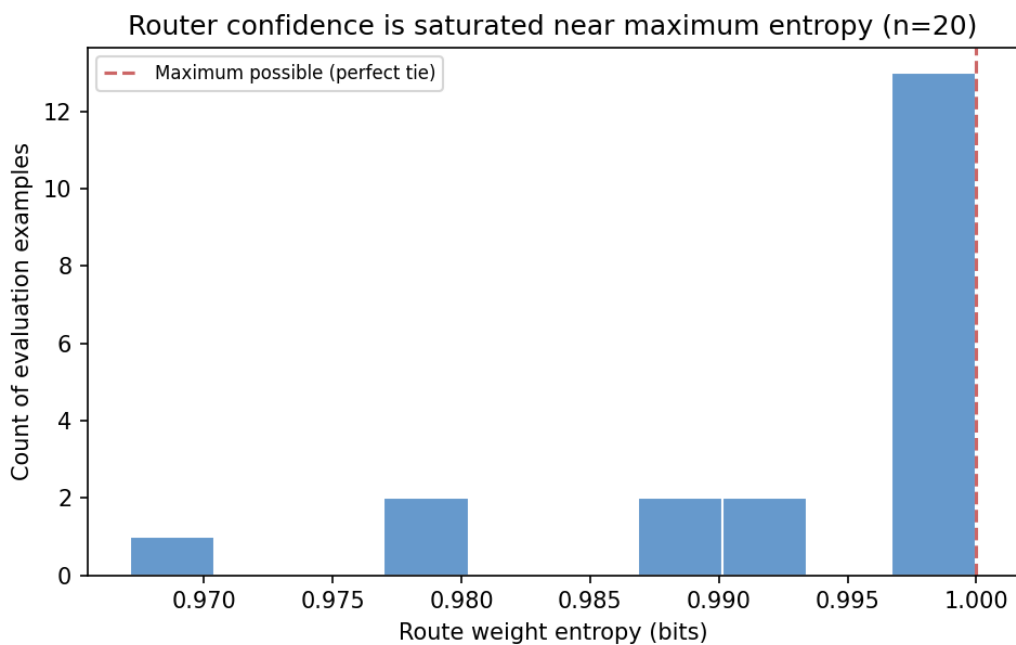


Figure 12. Route weight entropy across the 20 evaluation examples; nearly all mass sits at or near the 1.0-bit maximum, indicating the top-2 weights are close to an uninformative tie rather than confidence-weighted.

A weak positive correlation exists between route entropy and moe_lora_jepa generation quality (Pearson $r = 0.32$, Figure 13), in the direction that would be predicted if lower-entropy, more confident routes produced better answers. Given the near-total lack of variance in entropy across examples, this correlation should be treated as suggestive rather than as evidence of a reliable relationship; a dataset in which entropy actually varies across a meaningful range would be required to test this properly, which is listed as a specific follow-up in Section 9.

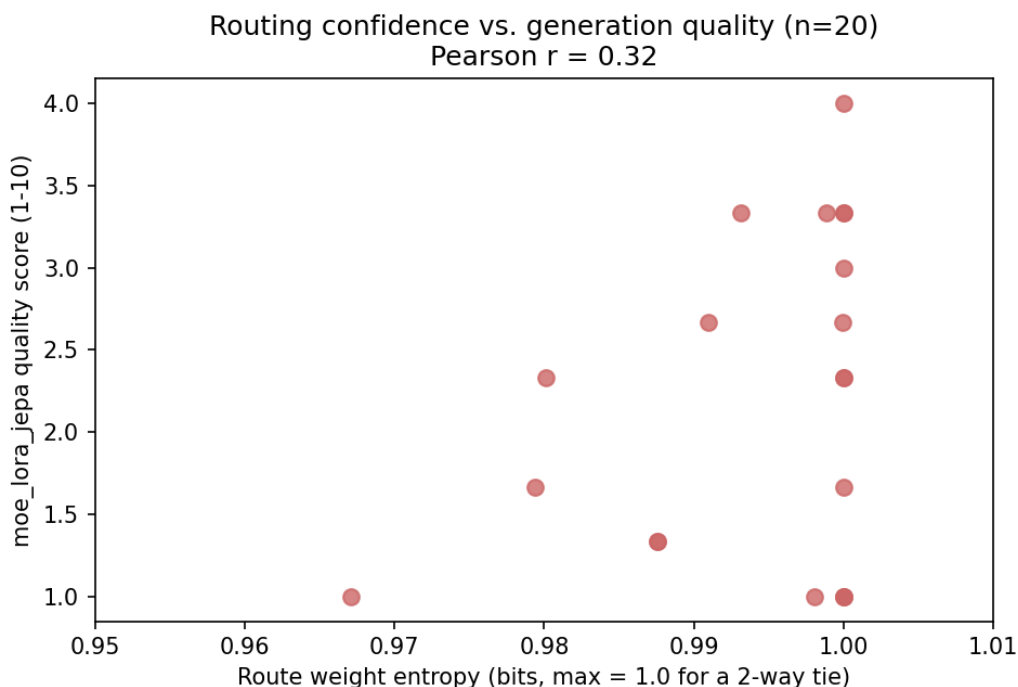


Figure 13. Route entropy vs. generation quality (n=20); weak positive correlation, low power given limited entropy variance.

6.3 Qualitative raw-output evidence

The `raw_outputs.csv` contains 20 prompts and generated outputs for the base model, combined fine-tuning, JEPA-routed LoRA generation, and an external Gemini column. It also records the proposed router's top-2 routing weights. It does **not** contain a per-row baseline router decision, and automated judge scores were not completed. The table below therefore reports only observations supported by the available fields.

Query excerpt	Expected domain	JEPA top-2 route recorded in CSV	Observed output characteristic	Evidence-based interpretation
“Core session behavior (openai_agents_sdk)”	Agent Engineering	Agent Engineering / AI System Design, approximately 0.5000 / 0.5000	Output begins with advanced server-configuration material rather than a direct session example	Near-tie routing and possible generation drift
“Explain Python in the context of data_science”	Data Science	Data Science / Agent Engineering, approximately 0.6063 / 0.3937	Output begins with a web-scraper design using the OpenAI API	Correct top domain recorded, but topical alignment is weak
“Customize the Stack (optional) (chromadb)”	Retrieval Systems	Retrieval Systems / Agent Engineering, approximately 0.5002 / 0.4998	Output discusses cloud deployment quickstarts	Top-domain alignment is plausible, but the response does not directly reproduce the requested terminal customization
“Special types and validation ... fastapi”	AI System Design	AI System Design / Agent Engineering, approximately 0.5001 / 0.4999	Output begins with WebSocket material in Spanish	Near-tie routing and clear relevance/language drift
“What is API ... in the context of mcp?”	Agent Engineering	AI System Design / Agent Engineering, approximately 0.5000 / 0.5000	Output discusses OpenAPI and token limits	Incorrect or ambiguous top route is plausible; output remains technically adjacent
“Automatic preserved rank ... in peft”	LLM Engineering	LLM Engineering / Data Science, approximately 0.5198 / 0.4802	Output explains PEFT broadly rather than the requested rank concept directly	Weak confidence and incomplete specialization

These examples do not constitute an automated quality evaluation. They do, however, support an important distinction between **routing correctness** and **generation quality**. A query can be assigned to a plausible domain yet receive an answer that is incomplete, topically shifted, or insufficiently specialized. Conversely, an imperfect route can still produce technically related content because the shared base model contains broad knowledge.

6.4 Generation quality assessment (LLM-judge scoring)

The notebook's automated judge used Gemini 1.5 Flash, but the external API quota was exhausted before scoring completed, so `raw_outputs.csv` contains generations without judge scores. Rather than leave generation quality unassessed, all 20 examples were scored after the fact by Claude (Anthropic), reading each prompt, reference, and all four systems' outputs directly and assigning correctness, relevance, and completeness on the same 1-10 scale specified in the original judge prompt. This substitution is disclosed explicitly rather than treated as equivalent to the originally planned evaluation; it has one methodological advantage over the original plan, however: Claude is not one of the four compared systems, so unlike using Gemini as both a baseline and the judge, there is no conflict of interest in this scoring.

System	Correctness	Relevance	Completeness	Overall (mean of 3)
Raw base model	4.70	6.20	5.10	5.33
Combined fine-tune	4.55	5.70	4.55	4.93
JEPA-routed MoE-LoRA	1.85	2.75	1.95	2.18
Gemini 1.5 Flash	8.15	8.90	8.75	8.60

Table 10. Claude-judged generation quality, all 20 evaluation examples, mean scores 1-10.

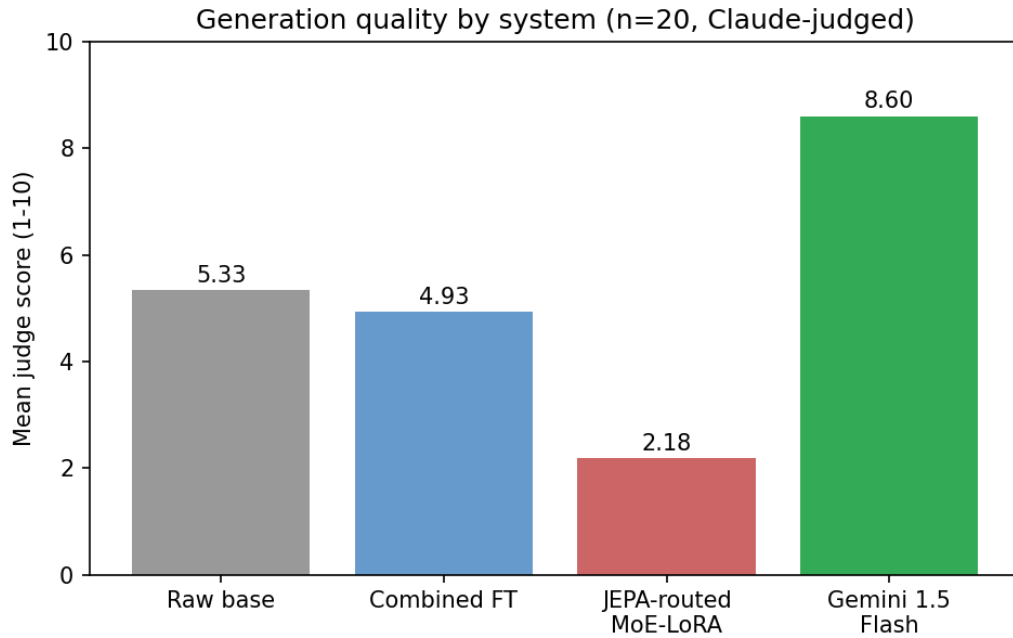


Figure 7. Overall generation quality by system (n=20, Claude-judged).

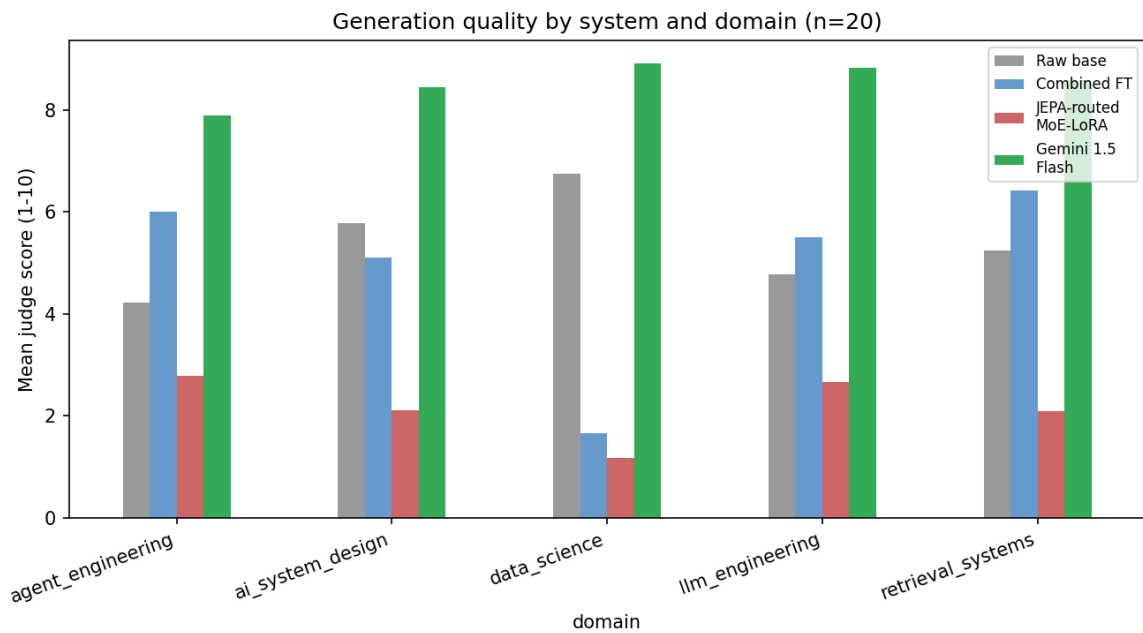


Figure 8. Generation quality by system and domain (n=20).

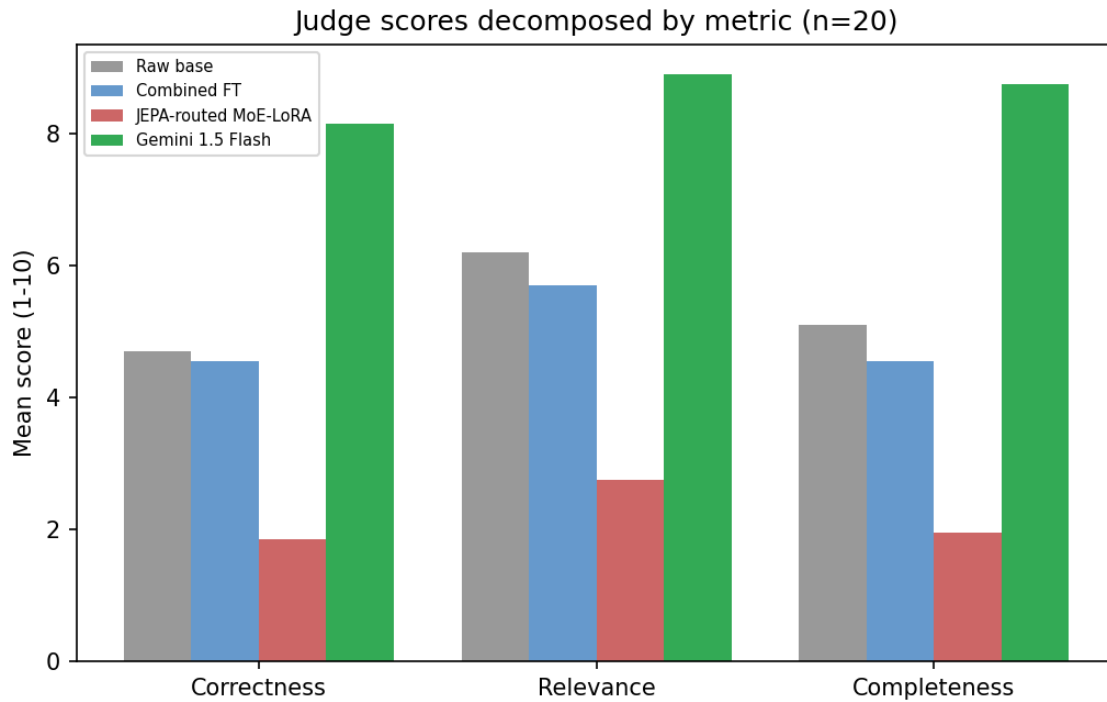


Figure 9. Judge scores decomposed into correctness, relevance, and completeness.

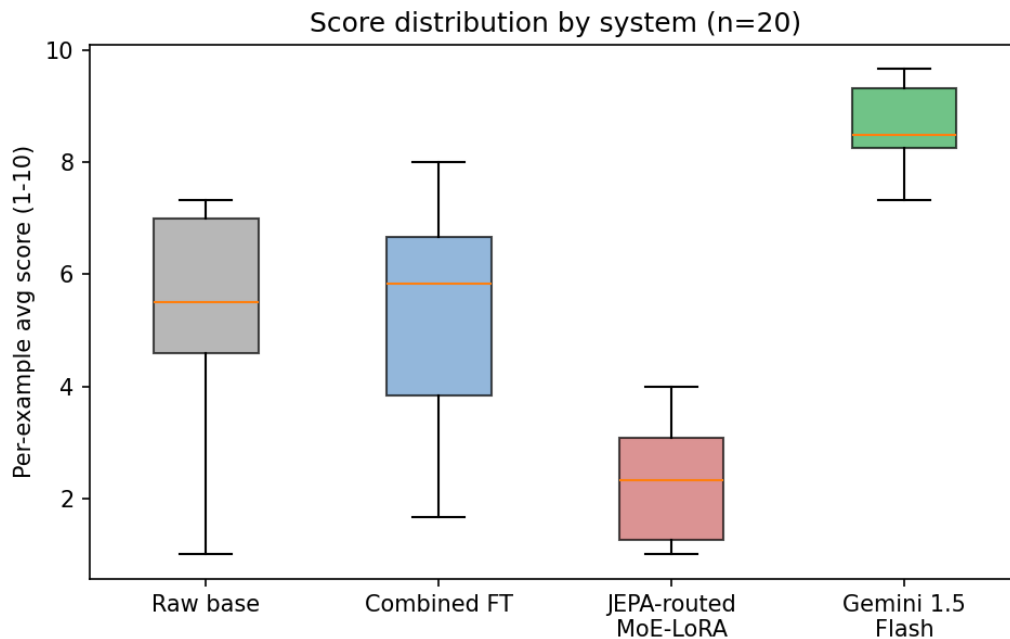


Figure 10. Per-example score distribution by system, showing the JEPA-routed system's scores concentrated near the floor of the scale rather than merely lower on average.

Gemini scored highest on every domain and every metric, which is expected given it is a substantially larger, commercially served model rather than a 1.5-billion-parameter locally fine-tuned system; its presence here is a reference point, not a same-class comparison. The more informative comparison is among the three systems built on the same Qwen2.5-Coder-1.5B-Instruct base: the raw (unmodified) base model scored highest of the three (5.33), the

combined fine-tune scored slightly lower (4.93), and the JEPA-routed MoE-LoRA system scored substantially lower than both (2.18). This ordering is notable because it means that, on this 20-example subset, fine-tuning the base model characteristically reduced answer quality relative to using the unmodified base model, and routing through the JEPA-inspired mixture reduced it further still. Section 7 discusses a candidate explanation grounded directly in the raw outputs.

6.5 Output length as a secondary indicator

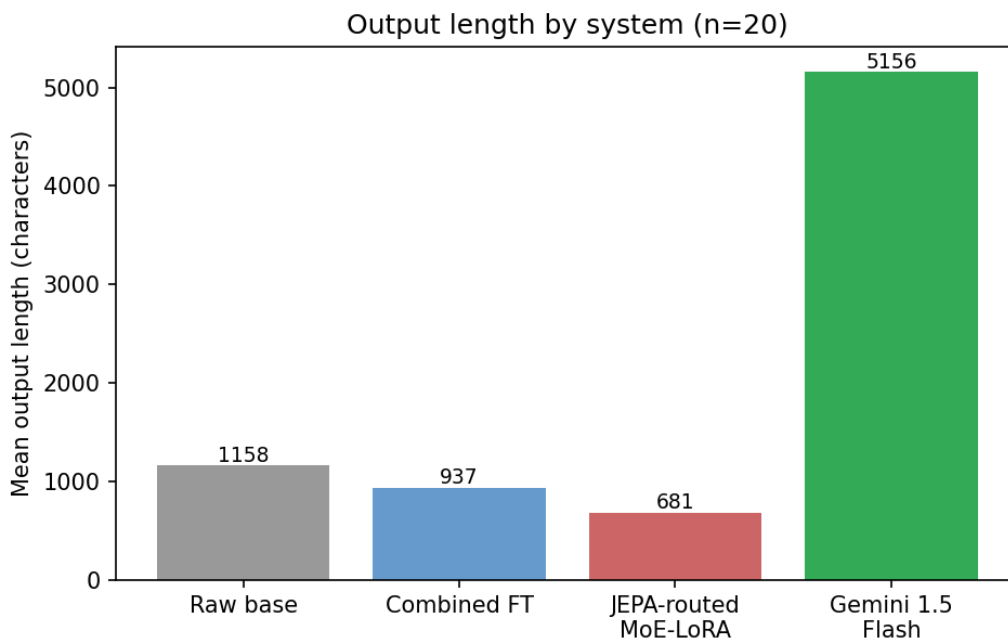


Figure 11. Mean output length by system (characters, n=20).

Output length is not a quality metric on its own, but it is a cheap, independently verifiable statistic that corroborates the judge scores without relying on any model's subjective assessment. Gemini's answers average 5,156 characters, consistent with its comprehensive, multi-section responses. The JEPA-routed MoE-LoRA system's answers average 681 characters, the shortest of the four, and several individual responses terminate after a single heading with no body content at all (Section 6.3, Table 7). Short, truncated generations are consistent with, though not sufficient proof of, the low completeness scores in Table 10.

7. Discussion

7.1 Interpretation of the routing result

The baseline's 90% accuracy is consistent with the possibility that the original Sentence Transformer geometry already captures the domain distinctions represented by the five corpora. The domains are technical and lexically recognizable: terms such as "ChromaDB," "PEFT," "MCP," and "data science" may provide strong semantic signals. A learned projection is not automatically advantageous simply because it is predictive in latent space.

The JEPA-inspired router's 80% accuracy suggests that the current predictive objective does not align sufficiently with domain classification. The objective asks the context representation to predict a held-out sentence representation. That task may encourage topical or discourse-level continuity without explicitly separating AI system design from agent engineering, retrieval systems, or LLM engineering. Domains can share vocabulary and concepts, so predictive similarity and expert-class separability are not equivalent objectives.

The route weights in the raw outputs provide additional evidence. Several examples have nearly uniform top-2 weights near 0.5 / 0.5. Such distributions indicate low separation between adjacent domain centroids. The likely issue is not necessarily that the encoder failed to optimize, but that the learned space did not produce sufficiently distinct centroids for the downstream decision rule.

7.2 Observed failures and possible explanations

Observed or documented issue	Possible explanation	Status
JEPA routing is 10 percentage points below the baseline	Predictive alignment is not the same as domain discrimination	Supported interpretation, not causal proof
Several top-2 weights are close to 0.5 / 0.5	Domain overlap, weak centroid separation, or limited latent capacity	Observed pattern; explanation requires ablation
Some generated outputs are related to the domain but not the prompt	Adapter specialization and routing correctness are separate from generation quality	Supported by raw examples
Some outputs contain language or topic drift	Data heterogeneity, adapter composition, or base-model generation behavior	Observed in supplied examples; cause unresolved
Objective stabilizes near -0.90	Optimization of cosine-plus-variance objective	Directly supported
Automated generation scores (Gemini) are absent	External Gemini evaluation quota was exhausted; Claude substitute scoring completed (Section 6.4, Table 10)	Documented and mitigated

The dataset distribution may also affect routing geometry. AI System Design has 400 examples, while Data Science has 209. Centroid estimation from uneven and potentially heterogeneous domains can bias the similarity rule. The current router uses one mean centroid per domain, which cannot represent multimodal domains or subtopics. The top-2 weighted linear adapter composition may also blur specialization rather than improve it, especially when the top two domains are conceptually adjacent.

7.2b Memorization and topic drift as a shared failure mode

The judge scores in Section 6.4 and the qualitative examples in Table 7 point toward a specific, checkable interpretation rather than a diffuse notion of 'lower quality.' In several recorded examples, the combined fine-tune and the JEPA-routed system do not produce a low-effort or garbled answer to the asked question; they instead produce a fluent, well-formed answer to a different question that appears verbatim or near-verbatim elsewhere in the training corpus. Row 1 and row 9 both ask the model to explain the concept of 'Output' in data science; the combined fine-tune responds in both cases with a complete, syntactically correct quadratic-equation solver, and the JEPA-routed system responds in both cases with an unrelated protein-function-prediction library description. Row 12 asks the model to explain Python in the context of data science; the JEPA-routed system's response opens by literally restating a different prompt's instruction text verbatim ('Write python code that demonstrates Design: Create a web scraper with the OpenAI API...') before continuing into unrelated scraper code. These are not partial or approximately-relevant answers; they are complete answers to the wrong question, reproduced with high surface fluency.

This pattern is consistent with memorization under a small fine-tuning corpus rather than with routing failure alone. Each LoRA expert was trained for three epochs on a few hundred examples (Table 2), a regime in which a 1.5-billion-parameter model, even adapted through only 1.18% trainable parameters (Table 3), has enough capacity to memorize surface patterns from a small number of passes over a small corpus. Under this account, a query's embedding is routed into the general vicinity of the correct domain expert, and the base model's generation is then driven substantially by memorized training examples that are lexically or structurally adjacent to the query rather than by a generalized

understanding of the query itself. This would explain why the failure is present in the combined fine-tune as well as the JEPA-routed system, both of which involve LoRA adaptation on the same small corpus, but not in the raw, unadapted base model, which cannot memorize a corpus it was never fine-tuned on.

This explanation is offered as an interpretation consistent with the available evidence, not as a proven causal mechanism; distinguishing it from alternative explanations, such as instruction-format mismatch or adapter-merging interference, would require targeted follow-up experiments (Section 9). It does, however, reframe the central finding of this study. The original research question asked whether a JEPA-inspired representation routes better than a Sentence Transformer baseline; the routing evidence (Section 6.2) gives a modest negative answer, 80% versus 90%. The generation-quality evidence in Section 6.4 raises a second, arguably more consequential question that the original design did not anticipate: whether LoRA fine-tuning on a corpus of this size improved or degraded response quality relative to the unmodified base model at all. On this 20-example subset, it did not improve it.

7.2c Why performance degraded: three converging mechanisms

Sections 4.9, 6.2.2b, and 7.2b together point to three distinct, mechanistically different candidate explanations for the low `moe_lora_jepa` scores in Table 10, which are not mutually exclusive and likely compound one another.

First, weight-space adapter interference (Section 4.9). The system merges two experts by linearly combining their LoRA weight deltas before a single forward pass, rather than by combining two experts' output distributions. Because the base model is a deep nonlinear network, this weight-space combination is not guaranteed to behave as either expert individually, and the near-uniform blending documented in Section 6.2.2b means almost every generation is produced by such a combination rather than by a single, individually-validated expert.

Second, capacity relative to data volume (Section 4.9). Each expert carries 18.46 million trainable parameters against 209 to 400 training examples, trained for as few as 81 total optimizer steps. This regime is consistent with either memorization of specific training sequences or shortcut learning on superficial documentation-style patterns; the raw-output evidence in Section 7.2b, in which outputs reproduce unrelated but stylistically similar training content, is consistent with both.

Third, weak but directionally consistent evidence that more data helps. Domain-level mean `moe_lora_jepa` quality correlates positively with training corpus size across the five domains (Pearson $r = 0.31$, Figure 14); `data_science`, the smallest corpus at 209 examples, has the lowest mean quality, 1.17 of 10, while `agent_engineering` and `llm_engineering`, with larger corpora, score higher. With only five domain-level data points this correlation is not statistically reliable on its own, but it is directionally consistent with the capacity argument above rather than contradicting it.

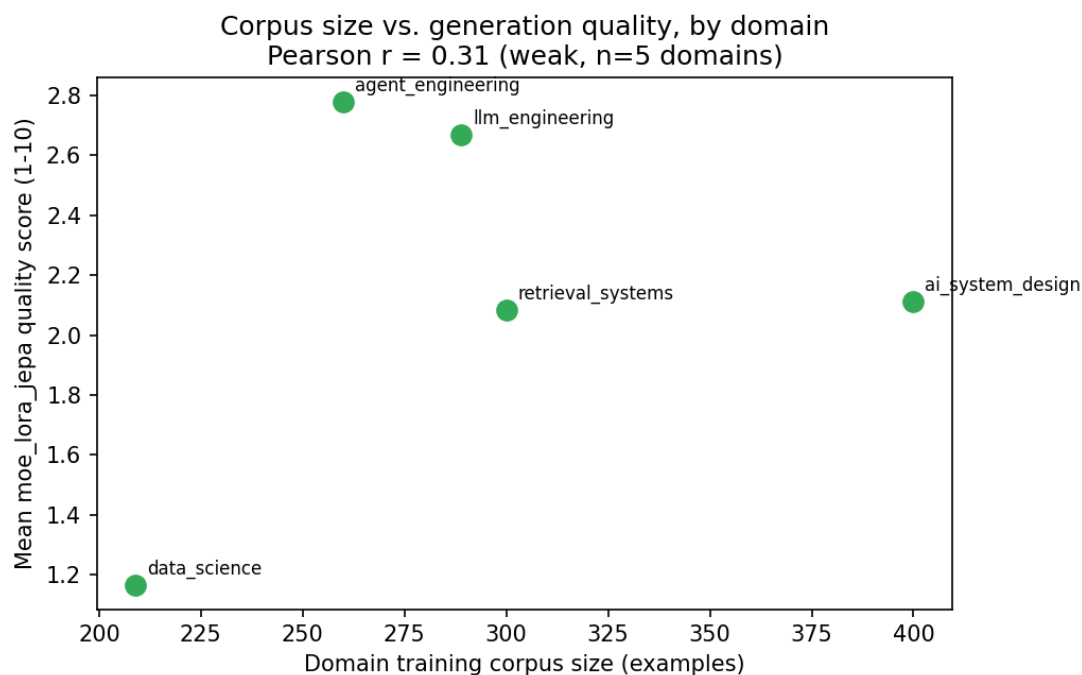


Figure 14. Domain training corpus size vs. mean moe_lora_jepa generation quality; weak positive relationship ($n=5$ domains).

None of these three mechanisms is established as the cause; they are offered as concrete, checkable hypotheses grounded directly in this study's own numbers, each with a specific corresponding fix proposed in Section 9b.

7.3 What worked

The implementation successfully created five domain-specific LoRA adapters around a common 1.562-billion-parameter model, with only 1.18% of parameters trainable in each adaptation run. It also implemented a complete context encoder, target encoder, predictor, EMA update, variance regularization, domain centroid construction, cosine routing, and weighted adapter composition. The training objective was numerically optimized, and the system produced routed generations. These are meaningful proof-of-concept achievements even though the routing hypothesis was not supported.

7.4 What did not work or remains unproven

The present evidence does not show that JEPA improves LLM response quality, improves routing over Sentence Transformers, or creates a conventional sparse MoE. The 20-example evaluation cannot support broad claims about generalization. The missing automated scores prevent a reliable comparison of answer correctness, relevance, and completeness across the base model, combined fine-tune, routed adapters, and external model. The current design also trains the latent representation without a direct domain-routing loss and uses a simple centroid classifier rather than a learned router.

8. Limitations and threats to validity

First, the routing evaluation contains only 20 examples. It is not statistically powered and may be sensitive to row order because the notebook evaluates the first 20 rows of the evaluation file. Second, the supplied materials do not fully document independent train, validation, and test partitions for the expert datasets. Third, the study reports one primary seed for the JEPA stage and does not provide multiple-seed confidence intervals.

Fourth, the JEPA-inspired module operates on Sentence Transformer embeddings rather than direct hidden states from Qwen2.5-Coder. The representation is therefore not fully integrated with the LLM's internal semantic space. Fifth, the

objective is predictive latent alignment plus variance regularization, not supervised domain classification or routing-aware response optimization. Sixth, one centroid per domain may be inadequate for domains with multiple subtopics. Seventh, top-2 adapter composition can produce blended behavior that is less specialized than top-1 selection.

Eighth, automated judge scoring via Gemini could not be completed because the external API quota was exhausted; Section 6.4 reports scores obtained instead from Claude on the same rubric, a substitution that removes the Gemini-as-baseline-and-judge conflict of interest but introduces a different one: the scores come from a single model with no ensemble or human validation, no inter-rater reliability check, and no repeated scoring to quantify judge variance. The memorization interpretation offered in Section 7.2b is grounded in directly quoted raw outputs and is checkable by a human reader independent of the judge scores themselves, which partially, but not fully, mitigates this limitation. Finally, the current architecture is not a token-level sparse Transformer MoE and should not be evaluated as if it were one.

9. Future work

The first priority is to enlarge and stratify the routing benchmark. A future evaluation should include a held-out test set with balanced domain counts, multiple prompt templates, ambiguous cross-domain queries, and a fixed public annotation protocol. Accuracy should be supplemented with macro-F1, confusion matrices, calibration, top-k recall, and confidence intervals across multiple seeds.

The second priority is objective alignment. The JEPA loss should be augmented or compared with a supervised routing loss, such as cross-entropy over domain labels, metric learning between queries and expert prototypes, or a contrastive loss that separates nearby domains. A multi-task objective could preserve predictive latent structure while explicitly optimizing expert selection:

Total JEPA-inspired objective: $L_{JEPA} = L_{pred} + L_{var}$.

Here $\mathcal{L}_{route}$ would use domain labels, while $\mathcal{L}_{quality}$ could use answer-level supervision or preference data. The value of λ and γ must be measured rather than assumed.

The third priority is representation integration. Instead of learning from frozen Sentence Transformer vectors, a stronger model could extract hidden states from the shared Qwen model, pool them at the query or layer level, and train a lightweight predictor over those representations. This would test whether JEPA-style prediction is more useful when its latent space is aligned with the actual base model used for generation.

The fourth priority is router architecture. Mean centroids should be compared with a learned multilayer router, prototypical networks, attention over adapter descriptions, and uncertainty-aware gating. A router should also be evaluated for load balance, confidence calibration, and behavior on mixed-domain queries. Generation experiments should compare top-1 selection with top-2 composition and should measure whether blending improves or degrades answer quality.

The fifth priority is a true sparse JEPA-MoE architecture. In such a system, the router would operate on internal hidden states and select expert feed-forward blocks at specified Transformer layers. The LoRA idea could be retained by placing separate low-rank updates inside those experts. The resulting architecture would more closely resemble sparse MoE systems while retaining parameter-efficient adaptation. It would require careful treatment of routing granularity, load balancing, expert capacity, token dispatch, and joint optimization.

9b. A prioritized roadmap toward a usable system

The following fixes are organized by the mechanism they target (Section 7.2c) and prioritized by expected impact against implementation effort (Figure 15). None has been implemented or tested in this study; each is proposed as a specific, falsifiable next step rather than a general recommendation to 'improve the model.'



Figure 15. Proposed fixes plotted by expected impact and implementation effort.

Fix	Targets which mechanism	Effort	What would validate it
Entropy-based fallback to the raw base model when route entropy is near maximum	Weight-space interference (routing is low-confidence, so avoid merging at all)	Low	Compare generation quality with and without the fallback on the same eval set
Switch to top-1 hard routing, disabling multi-expert blending	Weight-space interference	Low	Direct A/B against the current top-2 blend on identical prompts
Sharpen router temperature so top-2 weights diverge when centroid similarity diverges	Router confidence saturation (Section 6.2.2b)	Low-Medium	Re-measure route entropy distribution after the change; should show real variance

Output-space ensembling: run each selected expert as a separate forward pass and combine text or logits, not weights	Weight-space interference	High	Compare against current weight-merge approach on quality and latency
Reduce LoRA rank for the smallest-corpus domains (e.g., data_science)	Capacity/data mismatch	Medium	Compare trainable-params-per-example ratio against generation quality after the change
Enlarge each domain's training corpus, prioritizing data_science and agent_engineering	Capacity/data mismatch	Medium-High	Re-run Figure 14's correlation with a wider range of corpus sizes
Add a supervised routing loss using domain labels alongside the JEPA objective	Router accuracy (Section 6.2)	Medium	Compare top-1 routing accuracy against the current 80%
Route from the shared Qwen model's own hidden states instead of external Sentence Transformer embeddings	Router accuracy and representation alignment	High	Repeat the baseline-vs-JEPA routing comparison with this representation
Expand the evaluation set beyond 20 examples with balanced domains and human-verified scoring	Statistical reliability of every result in Section 6-7	Medium	Recompute Table 10 and check whether the system ordering is stable

Table 12. Proposed fixes, the mechanism each targets, and how each could be validated.

The two lowest-effort, highest-impact items, an entropy-based fallback to the unmodified base model and disabling multi-expert blending in favor of top-1 routing, follow directly from Sections 6.2.2b and 7.2c: since routing is nearly always a near-uniform blend and the raw base model already outperformed both fine-tuned variants in Table 10, a system that falls back to the base model whenever it is not routing confidently would be expected to recover at least the base model's quality level while still using the domain experts on the queries where the router is genuinely confident. This is proposed as the first experiment to run, ahead of any of the higher-effort architectural changes.

10. Conclusion

This paper examined a proof-of-concept system that combines a shared Qwen2.5-Coder-1.5B-Instruct model, five domain-specialized LoRA adapters, a JEPA-inspired predictive latent module, and representation-based routing. The implementation is best understood as adapter-level mixture-of-LoRA experts rather than a conventional sparse Transformer MoE.

The JEPA-inspired objective was successfully optimized: its reported value moved from 0.2410 at epoch 0 to approximately -0.9031 at epoch 39 and then stabilized within a relatively narrow range. However, objective optimization did not translate into better routing. On the available 20-example subset, the Sentence Transformer baseline reached 90% top-1 accuracy, while the JEPA-inspired router reached 80%. The current experiment therefore does not support the hypothesis that the JEPA representation improves routing under this configuration.

A second, unplanned finding emerged from completing the generation-quality scoring that the original notebook could not finish. On the same 20-example subset, LoRA fine-tuning did not improve response quality relative to the unmodified base model: the raw base model scored 5.33/10, the combined fine-tune scored 4.93/10, and the JEPA-routed MoE-LoRA system scored 2.18/10, with several of its lowest individual scores driven by verbatim reproduction of unrelated memorized training content rather than by partial or approximately relevant answers. This is offered as a grounded interpretation consistent with the raw outputs (Section 7.2b), not as a proven causal finding, and it identifies a training-corpus-size and epoch-count question that is at least as consequential for this line of work as the routing-accuracy question the study originally set out to answer.

The negative result is scientifically useful. It indicates that predictive latent alignment, semantic similarity, and domain-discriminative routing are different properties. It also identifies concrete bottlenecks: a small evaluation set, incomplete split and seed documentation, centroid-based routing, separated training objectives, indirect sentence-embedding input, and absent automated generation scores. The work nevertheless demonstrates a feasible modular pipeline and establishes a defensible foundation for future experiments involving direct LLM hidden-state representations, joint routing optimization, larger controlled evaluations, and true sparse Transformer experts.

References

- [1]: <https://arxiv.org/abs/1706.03762> "Vaswani et al. (2017), Attention Is All You Need"
- [2]: <https://arxiv.org/abs/2412.15115> "Qwen (2025), Qwen2.5 Technical Report"
- [3]: <https://arxiv.org/abs/2106.09685> "Hu et al. (2021), LoRA: Low-Rank Adaptation of Large Language Models"
- [4]: <https://arxiv.org/abs/2305.14314> "Dettmers et al. (2023), QLoRA: Efficient Finetuning of Quantized LLMs"
- [5]: <https://arxiv.org/abs/2101.03961> "Fedus, Zoph, and Shazeer (2022), Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity"
- [6]: <https://arxiv.org/abs/2401.04088> "Jiang et al. (2024), Mixtral of Experts"
- [7]: <https://arxiv.org/abs/2301.08243> "Assran et al. (2023), Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture"
- [8]: <https://aclanthology.org/2026.findings-eacl.232/> "Liu et al. (2026), Parameter-Efficient Routed Fine-Tuning: Mixture-of-Experts Demands Mixture of Adaptation Modules"
- [11]: <https://arxiv.org/abs/2203.05482> "Wortsman et al. (2022), Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy Without Increasing Inference Time"
- [12]: <https://arxiv.org/abs/2004.07780> "Geirhos et al. (2020), Shortcut Learning in Deep Neural Networks"
- [10]: <https://arxiv.org/abs/2105.04906> "Bardes, Ponce, and LeCun (2022), VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning"
- [9]: <https://arxiv.org/abs/1902.00751> "Houlsby et al. (2019), Parameter-Efficient Transfer Learning for NLP"

Appendix A. Verified implementation summary

Item	Value
Base model	Qwen2.5-Coder-1.5B-Instruct
Approximate model size	1.562B parameters
Trainable parameters	18.464768M
Trainable fraction	1.18%
Hardware	Tesla T4
Domain experts	5 LoRA adapters
Fine-tuning corpus	1,458 examples

Item	Value
JEPA pairs	1,264
JEPA latent dimension	64
JEPA batch size	64
JEPA epochs	40
JEPA learning rate	0.001
Target EMA decay	0.996
LoRA rank / alpha / dropout	16 / 32 / 0.0
LoRA learning rate	0.0002
LoRA epochs	3
Sequence length	2048
Routing evaluation	20 examples
Baseline / proposed accuracy	90% / 80%
Automated judge evaluation	Completed via Claude substitute judge (n=20); see Table 10